

session_orchestrator

session_orchestrator

Revision: 2 **Last modified:** 2026-07-08T06:55:46Z **License:** MIT **Status:** early scaffold — buildable increments: the alias-health registry + `is_operable` predicate, the flowing-pool claim registry (exactly-once, deadlock-free, single-owner), and the non-failover scheduler (assignment/placement). The same-session failover/resume spine is **NOT** implemented (its cross-config-dir `claude --resume` continuity premise is `UNCONFIRMED`: pending a POC).

A **project-agnostic, fully decoupled** Go engine for coordinating a *flowing pool* of session aliases behind a *floating orchestrator role*, with a no-fail-open health predicate that decides which alias may do work right now.

The model (flowing pool + floating orchestrator role)

- **Flowing pool.** Every alias — first-party (native) and third-party (provider) — lives in one shared pool. No alias is reserved or locked for exclusive use.
- **Floating role.** The orchestrator is a *role* over that pool, not an alias removed from it. Track work and the orchestrator role draw from the same pool; the role is simply scheduled first, so it never starves.
- **Transient unpickability only.** An alias is unpickable only while it is *claimed* (single-owner) or *in cooldown*; it returns to the pickable set at its natural priority on release or cooldown expiry.

What this increment ships

The `alias` package:

- **Registry** — a concurrency-safe set of aliases registered **by name only** (never credential values). Each alias carries its class (native/provider), capability rank, stable index, health state, and cooldown expiry.
- **Classify** — maps one live probe result onto the health taxonomy, scanning the body so an `HTTP 200` that hides a plan-cap string (e.g. a monthly usage limit) classifies as capped, never healthy.
- **IsOperable** — the total, **fail-closed** predicate: an alias is operable only on positive captured evidence (`HTTP 200` + the `VERIFY_OK` token + no error body + a healthy classification + not in cooldown + not in a known-bad state). Every un-evaluable outcome yields `false`. There is no fail-open path.
- **SortByPriority / FirstOperable** — the deterministic priority order (native before provider, free before cooling, stronger before weaker, stable tie-break) and a first-operable selection that returns an explicit “none” rather than falling through to an unhealthy alias.

The `claim` package:

- **Registry** — the flowing-pool claim registry: an exactly-once, deadlock-free, **single-owner** binding of an alias (one CLI credential/session context) to at most one work-unit at a time. `TryClaim` is a non-blocking atomic compare-and-set (`GRANTED / GRANTED_EXISTING / DENIED`); `Release`, `Renew`, and `TTL` / dead-holder reaping keep the pool honest. Liveness is proven, never assumed.

The `scheduler` package:

- **Schedule** — the **non-failover** assignment layer. Given a priority-ordered set of work-units, the flowing alias pool, and the claim registry, it places each work-unit onto the highest-priority **operable** alias by claiming it exactly-once, so no two work-units ever share an alias (single-owner). It is a pure composition of `alias.IsOperable` (fail-closed) + `claim.TryClaim` (atomic single-owner CAS). A non-operable alias is **never** assigned; a work-unit with no claimable operable alias is returned **explicitly Unassigned** (never dropped, never double-assigned); a work-unit already holding a live claim keeps it idempotently.

The clock is injected, so assignment is deterministic. (The re-homing of a degraded work-unit onto a new alias is the WS-C float — the `UNCONFIRMED:` failover spine — and is deliberately **not** here.)

The `supervisor` package:

- **Supervisor** — the WS-D §2.3 **death-detection** watchdog. It monitors a set of entities (the floating orchestrator role and/or pool aliases), each with a consumer-supplied heartbeat/last-seen timestamp, and on each `Check(now)` classifies every entity **ALIVE / SUSPECT / DEAD** from two evidence sources: the heartbeat age against a configured liveness window, and an injectable liveness proof. The proof is **authoritative in both directions** (mirroring the claim registry): a proof of death declares DEAD even while the heartbeat is fresh; a proof of life keeps the entity ALIVE even past the heartbeat window. It is a **signal, not an action** — it emits an honest per-entity verdict + an append-only transition log and returns the DEAD set, but performs **no** recovery itself (reaping / re-homing is the caller's job). The clock is injected (`now` is passed to every `Check`), so classification is deterministic. (The same-session failover/resume spine it would trigger is the `UNCONFIRMED:` WS-C float and is deliberately **not** here.)

Decoupling contract

This engine hardcodes

no track, alias name, directory, threshold, or project string. A consumer registers its own topology at runtime; the engine reads it. An `Alias` holds **no** secret material — the caller performs the probe (passing any key via env/config, never on the command line) and hands the engine only the observable result.

Build & test

```
go build ./...
go test -race -count=3 ./...
```

Not yet implemented (honest boundary)

The same-session **failover/resume spine** (capture session id → detect limit → quiesce to a safe boundary → atomic swap → `claude --resume` on another alias) depends on an `UNCONFIRMED:` premise — cross-`CLAUDE_CONFIG_DIR` `claude --resume` continuity — that a scratch-session POC must confirm before any implementation. It is deliberately absent from this scaffold.